The Impact Of Pit On Security And Portability

Leif Pedersen
Department of Computer Science
University of North Dakota
Box 9015
Grand Forks, ND 58202-9015
bilbo@hobbiton.org

***Abstract*— Pit[15], [16] is a general-purpose language designed to be an alternative to C. Its expressive ability is a superset of C, and it easily interoperates with C and assembly language, making it no less suitable than C for low-level programming.**

Its additional features allow the programmer to easily guarantee that there are no buffer or integer overflow vulnerabilities by automating memory allocation, with some cost to efficiency. Manual memory allocation is still well-supported for when the cost to efficiency is too great or, for example, in parts of a kernel where automatic allocation is impractical.

The additional features also make it practical to write source files that are still portable between operating systems after they are compiled to object files, substantially reducing the amount of work required to move an application from one operating system to another.

These additional features are also more widely applicable; they provide an infrastructure for managing complex data structures and abstract data types easily. This paper focuses on the applicability of Pit to security and portability, as these are important issues that Pit can have a significant impact on.

I. Introduction

General-purpose programming languages, referring to languages designed for writing wide ranges of applications such as C, Perl, and their relatives, may be characterized by their style of memory management and level of abstraction. As a general trend, less abstract languages require the programmer to explicitly code memory management logic and statements express CPU instructions more directly with greater detail, while more abstract languages require less attention to memory management and represent the intent of the program more directly with less detail. There are, of course, many points of view on classifying languages, but for this discussion, the interesting trade-offs between general-purpose languages are manual or automatic memory management, representing CPU instructions directly or the intent of the algorithms, and representing more detail or less detail. For example, C represents the CPU's instructions almost directly and provides no automated memory management and requires more detail in the code, however Perl manages memory entirely automatically and usually represents complex algorithms in less code. The cost to using Perl is, of course, efficiency; in many cases, Perl code is so slow it cannot meet the user's requirements. This pattern of trade-offs frequently applies to comparisons between a low-level and a high-level language. This brief discussion may seem to imply a trade-off between automated memory management and expression of fine-grained detail, however even if this connection holds (and it may not) it does not exclude the possibility of creating a language that can express both sides of the trade-offs in adjacent statements. This duality is precisely what Pit implemented.

Unfortunately, most software does not divide as neatly as most languages into "low-level" and "high-level" where, for example, low-level applications cannot afford the cost of automated memory management incurred by high-level languages and high-level applications cannot afford the cost of extra effort in development time incurred by low-level languages. Most nontrivial software has some code that does not need to execute efficiently and therefore would benefit from a high-level language and some code that must execute quickly or for other reasons cannot afford the costs of a high-level language. Graphical programs are an easy example of this; code that deals with individual pixels and polygons for rendering images and animations benefits significantly from optimizing at the level of individual CPU instructions, while code that directs larger user interface widgets such as window placement and buttons benefits far more from greater abstraction.

It's possible to bridge this gap by using two languages to write a program, but this approach has disadvantages. One problem is it's difficult for a module written in one language to access data stored in memory by a module written in a different language because most languages store data in memory quite differently from each other. Perhaps more importantly, using two languages to implement the software creates an artificial boundary in the logic based on the capabilities of the two languages rather than based on a natural separation of concern that's convenient for the program's design. A better solution is to use a language that allows both approaches and allows fine-grained control over when each approach is applied. Pit uses variable declarations to determine which approach to use, which allows convenient yet detailed control.

Automated memory management typically found in more abstract languages has benefits beyond saving significant amounts of work; they also allow improvements to both security and portability. Automated memory management can entirely prevent some classes of security holes, such as buffer overrun[2] or integer overflow[5], [1] bugs, and dynamically typed variables can help portability by loosening the run-time bindings between functions written by different vendors. These two benefits are the focus of this research.

Pit is a compiled language derived primarily from C. While it is designed to be familiar to C programmers, the languages

are difference enough in coding style that it will probably always be a manual task to translate between C code and Pit code in a way that produces reasonably understandable code. The most important innovation is that it simultaneously supports the variable types found in C, called "primitive variables" and another class of variables called "auto variables". Auto variables are declared syntactically using the keyword "auto" in place of the type specification. Auto variables are similar in many respects to variables implemented by Perl; in this regard, Pit is secondarily derived from Perl. They are dynamically typed and manage memory automatically. The programmer can choose to write the entire program using only auto variables, only primitive variables, or a combination of both. This allows the programmer to choose to use automated or manual memory management with every variable declaration. Thus, the programmer can write some code that has all the characteristics of C code by avoiding auto variables, and in the same function write more code that has all the benefits of automated memory management and dynamically typed variables by using auto variables.

The compiler implements auto variables by inserting function calls to a supporting library. Each operation translates into a function call, and it inserts additional calls for incrementing and decrementing reference counts for garbage collection. It also transparently inserts calls to this library for converting from a primitive variable type to an auto variable or vice versa, making it easy to assign a value from a primitive variable to an auto variable and vice versa. Pit is self-hosting, meaning that it can express the compiler tools and supporting libraries for itself with no tools written in other languages (although this isn't implemented yet). To accomplish this, the supporting language library uses primitive variables to construct the data structures that store information for auto variables.

Pit is designed to be suitable for (but not limited to) low-level programming, such as for operating system kernels, device drivers, system libraries, etc. This is perhaps the class of software that is most starved for improvements in languages because most implementations of recent languages require a virtual machine, prevent manual memory management, and don't have a way to access assembler code. Pit can interact with C code reasonably easily, which makes transitioning or interacting with an existing project easier. All C variables map directly to primitive variables in Pit, and the calling convention for functions is close enough that a simple wrapper function can work around the differences.

II. Security

Even today, after so much study and work, cutting edge operating systems still have a dismal outlook on security. Even if all the bugs were fixed in existing code, new ones are constantly introduced with new code. For example, nobody reasonably expects the existing operating systems to ever be free of buffer overrun bugs until there are new innovations. Until then, the industry must settle for staying one step ahead of attackers. Operating system code is perhaps the most widely used code, since every computer needs it. This probably makes it the most targeted code for malicious attackers and probably the most widely audited code as well, but to date, nobody has produced a modern operating system that is free of security problems. OpenBSD is probably the operating system with the most proactive approach to security, and after many years it is still unusual for them to go even a few months without finding another buffer overflow bug[13]. Many good ideas have been developed to improve security for low-level code, and yet all have fallen short of complete success, where a successful approach must achieve a high level of confidence that the code has no buffer overrun bugs, no integer overflow bugs, and do so without a loss of efficiency. In general, when this paper refers to security, the scope is limited to memory allocation, buffer overrun, and integer overflow bugs because they are the bugs that this programming language can help prevent, and this class of bugs covers the overwhelming majority of security holes.[18], [9], [8]

C is the language of choice for operating system code for good reasons. Low-level programmers absolutely need easy access to in-line assembly code for hardware IO and setting special-purpose CPU registers. Also, they need to be able to write efficient code, as speed is a major point of competition between operating systems; this means that if there are any inefficient high-level features, they need to be optional. (C has no such features) A language that requires an interpreter or virtual machine is clearly not an option. Perhaps most importantly, programmers need to have a clear idea what the output of the compiler will be, because they are working in an environment where many CPU facilities are often unavailable. When the translation is too abstract or layered too deeply, a low-level programmer will often become frustrated because the compiler will generate unexpected output that may, for example, interfere with the kernel's memory management, use floating-point facilities when they are not available, or attempt to access variables that are temporarily unavailable. Complex tools lead to more problems than simple tools of equal power. To address this, source code that does not use auto variables will never be translated into machine code that automatically manages memory. Because the extra features have no impact on the output unless they are used, Pit works at least as well as C for low-level code. Pit fits all of these requirements, and provides a way to choose when to use automatic memory management as well. Pit also interacts more easily with C than most other languages. This makes it a good step forward from C with little cost.

Pit is by no means a silver bullet. The advantage is that the programmer gets to choose whether the benefit is worth the penalty, rather than being forced to write an entire module in a low-maintenance high-overhead language such as Perl or in a high-maintenance low-overhead language such as C. To effectively use Pit, a programmer must choose wisely between primitive and auto variables. This requires a balance of efficiency and maintainability. As long as a programmer only uses auto variables, there is an absolute guarantee that his code will not have these security bugs. However, this guarantee comes at a cost to efficiency that is often significant. On the other hand, every time a programmer uses primitive variables, he runs the risk of introducing security bugs. It is well-established that program execution time is related to

Zipf's Law. Specifically, a small amount of code accounts for most of the execution time of a program; a widely accepted estimate is that 10-20% of the code accounts for 80-90% of the execution time. (The sections of code that account for most of the execution time are the "time-critical sections" sections of code.) On the other hand, Zipf's Law does not apply to security. When considering execution time, a program can be somewhat less efficient, but a program cannot be somewhat less secure. It's either secure or not, because when an attacker discovers a security hole, typically the whole system is compromised. This leads to the conclusion that while all the code in a program must be secure, only about 10-20% of it needs to be efficient. It is easy to take advantage this in Pit. If only 10-20% of the code is written using primitive variables, then the risk of security holes is eliminated in 80-90% of the code. Additionally, this leaves programmers more time to carefully audit the remaining code for security bugs, both because they don't need to audit the other code for these bugs and because the other code was easier and faster to write initially. To benefit from this aspect of Pit, a programmer must to be willing to accept this trade-off and use it to his advantage. Most of the existing research for detecting security errors in C code or aborting a C program when a buffer overrun occurs is easy to adapt to Pit's primitive variables. Such an adaptation should typically assume auto variables have no such security errors and do the security checking only on primitive variables.

A. C comparison

```
#include <stdio.h>
#include <string.h>

int main() {
    char in[1024], *inptr, *num;
    int first, second;

    while(fgets(in, sizeof(in), stdin)) {
        inptr = in;

        if(! (num = strsep(&inptr, " ")))
            continue;
        first = atoi(num);

        if(! (num = strsep(&inptr, " ")))
            continue;
        second = atoi(num);

        printf("Sum is %i\n", first + second);
    }
    return 0;
}
```

Fig. 1. Example program in C.

Figure 1 shows a C program that adds pairs of numbers read from standard input. Figure 2 shows the same program written in Pit. Notice that in Pit, auto variables can manage all the array sizes and memory allocation, thus eliminating any potential for mismanaged string buffers. As shown in Figure 3, they can also be managed manually just as easily as in C, with the same risk to security. This is the trade-off between improving efficiency and improving security and maintainability.

While Figure 1 clearly has no buffer overflow vulnerabilities, there are several ways they could easily be introduced

```
import io;
import string;

public function(out int $exit_status) $.main = {
    private auto $in, $nums;

    // $io.stdin is a global symbol.  => is
    // the method-call operator.  readline
    // is the name of the method that reads
    // a line of input.  The line is stored
    // in $in and returned.  Dots are
    // simply part of a symbol's name
    // indicating its namespace.
    while($io.stdin=>readline(ret $in)) {
        // $string.split() is a function that
        // splits $in on " " and stuffs the
        // result into $nums as an array of
        // strings.
        $string.split($in, " ", $nums);

        // Convert the strings to integers.
        $string.2int($nums(0));
        $string.2int($nums(1));

        // The object $io.stdout has a method
        // called writef that is semantically
        // analogous to C's printf().
        $io.stdout=>writef(
            "Sum is $sum\n",
            hash {
                "sum", $nums(0) + $nums(1),
            },
        );
    }

    $exit_status = 0;
}
```

Fig. 2. Figure 1, rewritten in Pit.

either when the code is modified or as a simple miscalculation when writing it in the first place. For example, the wrong size could be passed to fgets(), a pointer error could be introduced where the string is parsed with strsep(), or an error could be introduced in the format string passed to printf. Also notice that the code sets an arbitrary limit on the length of an input line, and while it is obviously possible to work around this, doing so makes the program much more complex and adds more opportunities for security holes. For example, dynamically allocating just enough memory to hold the input can lead to accidentally freeing a dynamically allocated chunk of memory twice[6] or misusing realloc().

Figure 2 shows the same program, this time written in Pit using auto variables. Notice that neither the length of a string nor a pointer into the string are ever used, and the bounds on any indexing operations for auto variables are checked at run-time. All the sizes calculated automatically because memory is managed automatically. This means the opportunities for security holes that were present in Figure 1 are eliminated.

As an example of how to trade security for efficiency, Figure 3 shows the same program written in Pit, this time using some primitive variables. Although auto variables weren't completely eliminated here, they could be. In particular, $in is implicitly converted to an auto variable when passed to $string.split, and $nums remains an auto variable. Notice how easily primitive and auto variables can work together, allowing the programmer to choose precisely what code to optimize for efficiency at the risk of security and maintainability, and what code to optimize for security and maintainability at the cost

```
import io;
import string;

public function(out int $exit_status) $.main = {
    private string(char, 1024) $in;
    private int $in_len;
    private auto $nums;

    loop: {
        // Pit strings are _not_ null-
        // terminated.  $in_len is an in-out
        // parameter.  Going in it's the
        // maximum length of $in and coming out
        // it's the actual length of $in.
        $in_len = sizeof($in);

        // _p means primitive.  => is the
        // method-call operator.
        $io.stdin=>readline_p($in, $in_len);

        // readline_p modified $in_len
        unless($in_len) break;

        $string.split($in, " ", $nums);

        // Convert the strings to integers.
        $string.2int($nums(0));
        $string.2int($nums(1));

        $io.stdout=>writef(
            "Sum is $sum\n",
            hash {
                "sum", $nums(0) + $nums(1),
            },
        );
    }

    $exit_status = 0;
}
```

Fig. 3. Figure 1, with some speed optimizations.

of efficiency.

III. Portability

Portability refers to the effort required to transfer a program from one system to another. Another long-standing problem with C is programs usually must be distributed in source form, and the time-consuming process of compiling must be done by end-users. For large applications on a mid-range computer, this can literally take days of CPU time, to say nothing of the time required of people. This is necessary because at compile-time, a program is locked into a particular version of a particular operating system on a particular CPU. There are some systems to mitigate this; for example, the FreeBSD Ports Collection[11] is a repository of software that has already been patched as necessary to allow it to be easily compiled on FreeBSD. This is used to create precompiled packages that may be installed quickly by end-users. NetBSD and OpenBSD, as well as several distributions of Linux, have similar facilities for installing common software. These projects represent an enormous amount of ongoing maintenance, as each port must be individually updated whenever original author releases a new version and because these systems frequently break as a result of trivial differences in the C system library between systems. This often introduces an long delay before users can run new versions of their favorite software.

Unfortunately, the availability of such a porting system has become a major consideration when choosing an operating system. Otherwise superior operating system distributions may be abandoned because they lack strong support for ported software. Many software packages are simply unavailable for many operating system distributions. Complicating systems further, some operating systems implement awkward compatibility layers for software that is only distributed for other operating systems; for example, FreeBSD implements a compatibility layer for Linux software, but to use this for a typical Linux application, an additional Linux version of every shared library must be installed. Many programs are not available for FreeBSD at all, except by using an executable compiled for Linux on this compatibility layer.

While natively compiled code obviously cannot be portable between hardware architectures, it is not impractical to make object files portable between operating systems on the same architecture. To do so, the system library needs to provide a consistent interface to the operating system, as seen by object files, not just as seen by source code. (Here, interface refers to a set of function prototypes and global variables.) This interface must be extensible in a way that remains back-compatible at the level of machine-code.

The goal is to translate any application into object files just once for each hardware architecture, then distribute the object files to all operating systems for that hardware architecture. The installation procedure links the program and copies it into the appropriate place on the filesystem. While there will still be portability issues, most notably the locations of files in the filesystem, this will substantially reduce the effort required when porting software. System-specific configuration issues such as paths can be addressed in other ways. Typically, software installation should entail downloading the software distribution, linking the distributed object files with the system library and third-party libraries, and copying the executable and ancillary files into appropriate locations in the filesystem.

Traditionally, the number of times a program must be compiled can be described as the product n*m, where n is the number of hardware architectures and m is the number of operating systems. If object files are portable between operating systems, this is reduced to n, because the object files can be compiled once and distributed in compiled form. To do this, parts of the compilation that are dependent on the operating system must be resolved at link-time or be eliminated; resolving them before creating modules locks the modules into a target operating system. While it is unlikely that all portability issues will be resolved for all programs, this approach will significantly reduce the work required for most programs.

As mentioned above, the object files need a consistent interface to the operating system. To accomplish this, two important points must be considered:

- The system library cannot define preprocessor macros that are used in the application's modules, because the contents of these macros are almost certain to change. This is generally not a significant issue in Pit because it does not require a preprocessor.
- Variables passed between the system library and applications must be represented in a consistent way. Most importantly, this includes integers and structs.

```
#define EMIT_FOO_FLAG(a) \
    ((a) & 0x40 ? puts("True") : puts("False"))
```

Fig. 4. A preprocessor macro for a C library that causes dependent programs to lose portability when translated into an object files.

```
int emit_foo_flag(a) {
    if(a & 0x40)
        return puts("True");
    else
        return puts("False");
}
```

Fig. 5. A C function that does not have the difficulty described in Figure 4.

A. Preprocessor

C's preprocessor carries much blame for binary incompatibility. In C, a library can declare macros that the compiler resolves when the calling module is compiled, and on another operating system, this library may assign different values to these macros. Module interfaces should avoid preprocessor macros in favor of variables declared "constant" and function calls. For example, if the C header file fragment shown in Figure 4 were in a system library, it would cause object files compiled with it to be incompatible with any system that implements the contents of the macro differently (for example, by changing the value 0x40). Changing the preprocessor macro to a function (Figure 5) prevents the compiler from resolving the dependency at compile-time; rather, the dependency is resolved at link-time, and linking can be postponed until after the program is distributed.

Pit does not include a preprocessor. Preprocessor features are obviated in ways that avoid portability problems. There are three actions typically performed by the C preprocessor: substituting constant values, applying in-line calculations, and including header files. The first two are detrimental to portability and are trivial to accomplish with constant global variables and functions. In place of the third, Pit uses "interface files" to describe the symbols in a module. The compiler automatically generates an interface file when it translates the module the interface file describes. The calling module uses the "import" statement to reference the interface file. The import statement is roughly equivalent to the "#include" statement in C's preprocessor. (The only substantial difference is that the compiler processes import statements in the same pass as the rest of the code.) While no preprocessor is bundled with Pit's compiler, it is easy to apply one if desired. As long as the effects of each preprocessor definition are confined to the software package they are defined in, there will be no adverse affect to portability.

B. Passing integers portably

```
int fseeko(FILE *stream, off_t offset, int whence);
```

Fig. 6. This C function prototype is unportable after translation because the size of off_t will vary between operating systems.

The other problem is passing variables to functions in the system library. This requires auto variables to solve. If a parameter to a function is represented as an auto variable, it can be extended without recompiling existing code because of the structural description auto variables include during runtime. For example, consider file IO functions. Rather than representing the offset into a file as a statically-sized integer as shown in Figure 6, it is represented as an auto variable, as shown in Figure 7. Clearly, this solution will sometimes involve a cost to efficiency. This cost is not as severe as it seems at first, however, because not all values need to be passed in an extensible way. For example, any value that represents something bounded by memory can be represented with an integer whose size matches the hardware's pointer size, referred to as a "naturally-sized integer". Pit guarantees that the size of "int" will always be naturally-sized, so there is no need for the "size_t" and "ssize_t" types provided by C. Thus, any integer representing, for example, an array index, array size, or the size of anything stored in memory can be represented with a naturally-sized integer. If the architecture is extended to a larger pointer size, for example with the recent 64-bit extensions to x86, all the code must be recompiled anyway because the size of a pointer will change. Naturally-sized integers represent most of the integers passed between functions, leaving only a few of them requiring the extensible representation provided by auto variables.

```
public function(in auto $file, in auto $offset,
    in char $whence) $seek prototype;
```

Fig. 7. This Pit function prototype is portable because $offset can be any size integer. At run-time, the system library will convert it to what the kernel requires. If the conversion would truncate the value, the system library throws an exception back to the caller. $whence is simply the letter b, h, or e, meaning the offset is relative to the "beginning", "here", or "end". This way, the the C preprocessor macros SEEK_SET, SEEK_CUR, and SEEK_END are unnecessary.

Most numbers that represent the size of an object that cannot fit in memory represent IO routines where the CPU speed is not the limiting factor. Going back to the file IO example, consider an integer that represents an offset into a file. A file's maximum size is dependent on the operating system, not the CPU architecture. In the not-so-distant past, file sizes were represented with 32-bit variables, limiting files to 4GB. At the time this was decided, it was wasteful to use larger values; an "industrial size" array of hard disks occupying many large machine rooms might approach 4GB. Today, the industry uses a 64-bit value for representing offsets into files, but it is unreasonable to rely on that being large enough forever, particularly with the proliferation of distributed storage systems such as RAID in the consumer market. In fact, Sun Microsystems has already introduced ZFS, a filesystem that supports disk arrays larger than 2^{64} bytes in size, and as very few operating systems currently ship with a system library that does not use a 64-bit integer for this, another transition will be necessary eventually. Standard C libraries address this by using off_t to represent file offsets. While this allows the size to be increased without modifying the source code to a program, this is still unsatisfactory, because it requires the compiler to generate different assembler code depending on the target operating system's definition of off_t. Simply compiling two

versions is infeasible, because other data types are likely to be in transition as well, each of which doubles the number of compiled versions required. A better solution is to abstract the type of a file offset out of the compiled code entirely (Figure 7). An auto variable can represent the integer without requiring a particular size; the amount of memory allocated by the run-time library will always be sufficient to represent the number.

While representing a file offset with an auto variable reduces efficiently, the only time a file offset is communicated to the system library is when seeking to a new location in the file or querying the size of the whole file. Both of these operations are heavily bounded in speed by their hardware IO requirements (in particular, seeking to a new location on the disk), not the CPU, so the cost should not be measurable. When reading or writing the file, the maximum amount that can be read is bounded by memory, so a naturally-sized integer is guaranteed to be sufficient when representing the amount read or written in a single call. For example, Pit's equivalents to C's read() and write() functions use a naturally-sized integer since they both require a buffer in memory, but the equivalents to lseek() and fstat() use autos to represent the file size. There may be examples of other system library calls that have not yet been analyzed where the cost to efficiency must be weighed against portability. If optimizing does not yield satisfactory performance for these examples, one possible solution is to provide two interfaces, one that uses auto variables for portability and one that uses fixed-sized primitive integers for speed.

When providing a portable interface, integers may be represented as primitive variables everywhere except where the parameter passes the interface. For example, there is no compelling reason as far as portability is concerned for the system library to use auto variables after receiving a parameter to a function call, because the internals of the system library may easily be changed without changing the API. Also, for situations where the value is otherwise bounded by the design of the caller, the caller may use primitive integers to represent the value. In these cases, the compiler will automatically promote the primitive integer to auto variable when setting up the parameters to the function call and immediately assign the value to a primitive integer in the system library. In this case, the inefficiency is minimized by minimizing the number of operations performed on the variable while it is represented as an auto variable. As long as the parameter passes the library's interface as an auto variable, the interface remains portable.

In short, while there is some cost to using auto variables to pass integers to libraries, the cost can be minimized to a point where it is negligible in most cases.

C. Passing structs portably

The final major difficulty with passing parameters to system library calls in an extensible way is the inflexibility of structs. To be portable, a struct must always have exactly the same format, meaning it must contain the same elements in the same order, each with the same type, with no additional elements. It is a long shot to decide on a format for a struct, and then simply hope that nobody will ever change it and that it will fulfill all future requirements. To do this for all the structs in the system library is clearly impossible. However, the format need not be so rigid, by contrast, if the library uses hashes (as a dynamic type assigned to an auto variable) in place of structs. With hashes, code can check for missing elements, the elements may be in any order, and new code may add additional elements defined after old code was compiled. For example, if a new version of the system library looks for a new element in a struct, it will simply see whatever uninitialized garbage happens to be in memory past the end of the struct. However with a hash, which elements are present is stored in a dynamic hash table, and the existence of the element is checked at run time. This allows the new system library to fall back to old behavior or assume a default value if a new element is missing. Old code or code compiled on different operating systems remains compatible without recompiling.

```
import io;
import socket;

public function(out int $exit_status) $.main = {
    private auto $s;
    $socket.new($s, hash {
        "network", "ip4",
        "transport", "stream",
        "laddress", "127.0.0.1",
        "raddress", "127.0.0.1",
        "rport", 22,
    });

    private auto $buf;
    $s=>readline($buf);
    $io.stdout=>write($buf);

    return 0;
}
```

Fig. 9. An equivalent Pit program to Figure 8. In Pit, the error handling may be omitted, as Pit uses exceptions for reporting unhandled errors gracefully. A hash is a robust and simple way to pass all the necessary parameters in one call. Notice that unlike a struct, the order of the elements in the hash does not matter, improving portability. All the parameters passed in the previous C example can be combined into a single hash, greatly simplifying the library interface. Also, either the integer 0x7f000001 or the string "127.0.0.1" may be specified as an IP address, because the library can check whether the auto variable is an integer or string and do the required conversion. As should have been done in C's libraries, endian conversions for port and network numbers are always done by the library – so there's no htons(). There's no need to initialize unused values because a hash keeps track of which values are initialized.

Programs written in Perl have had much success with portability, primarily due to the flexibility of Perl's hash data type compared with C's struct data type. Pit's auto variables support the same hash data type as an analog to the struct data type supported by Pit's primitive variables since they are logically quite similar; the only major defining difference is whether the names of the fields can change during run-time. This allows APIs for libraries written in Pit to take advantage of the same benefits to portability that Perl provides.

An excellent example where this is useful is the API for establishing a TCP network connection. This API was extended several times to support new networking protocols, wreaking havoc on portability. In C, the connect() system call requires the programmer to pass a struct containing the family ("AF_INET"), port, and address. Most define additional

```
#include <string.h>
#include <sys/types.h>
#include <unistd.h>

/* Network-related includes */
#include <arpa/inet.h>
#include <netinet/in.h>
#include <sys/socket.h>

int main() {
    struct sockaddr_in my_addr, their_addr;
    int sockfd;
    char buf[1024];
    ssize_t ssz;

    /* Prepare my_addr and their_addr, which are temporary structs passed to bind()
     * and connect().  Notice the very important, but easily forgotten call to
     * memset().  This is required because implementations may add optional fields,
     * and we must pass zero for these, not an uninitialized value. */
    memset(&my_addr, 0, sizeof(my_addr));
    my_addr.sin_family = AF_INET;
    my_addr.sin_addr.s_addr = inet_addr("127.0.0.1");

    memset(&their_addr, 0, sizeof(their_addr));
    their_addr.sin_family = AF_INET;
    their_addr.sin_addr.s_addr = inet_addr("127.0.0.1");
    their_addr.sin_port = htons(22);

    /* Open socket */
    if((sockfd = socket(AF_INET, SOCK_STREAM, 0)) == -1) {
        perror("socket");
        return 1;
    }
    if(bind(sockfd, (struct sockaddr *)&my_addr, sizeof(my_addr))) {
        perror("bind");
        close(sockfd);
        return 1;
    }
    if(connect(sockfd, (struct sockaddr *)&their_addr, sizeof(struct sockaddr))) {
        perror("connect");
        close(sockfd);
        return 1;
    }

    memset(buf, 0, sizeof(buf));
    if((ssz = read(sockfd, buf, sizeof(buf))) == -1) {
        perror("read");
        close(sockfd);
        return 1;
    }
    if(write(STDOUT_FILENO, buf, ssz) < ssz) {
        perror("write");
        close(sockfd);
        return 1;
    }
    if(close(sockfd)) {
        perror("remote close");
        return 1;
    }
    if(close(STDOUT_FILENO)) {
        perror("stdout close");
        return 1;
    }

    return 0;
}
```

Fig. 8. A C program for connecting to the local system on port 22 and reading one packet. A well-designed API may improve this, but it would not change the difficulties with portability.

padding in the struct that reserves space used for operating system specific parameters or other protocols such as IPv6. This space must be zeroed for portability, as shown in Figure 8. All this adds up to an interface that is easier to use wrong than to use right, and is not at all portable after compilation. This is not simply bad library design. The interface would probably be reasonably easy to use and extend if C provided an extensible representation of data types.

Consider the Pit alternative in Figure 9. Here the information is better represented as a hash so the system library can check for unspecified fields (rather than relying on the caller to zero them or reading uninitialized memory), and the system library can take an appropriate default action because all the information about the hash is encapsulated into an extensible self-describing format, even at run-time. In this example, both the local and remote IP address and port are passed in the same call. If the local IP and port are omitted, the system library will simply ask the kernel to automatically select reasonable

values. If the remote IP and port are omitted, the system will create a listening socket.

With these portability issues resolved and a reasonably well-designed standard for the system library, object files should be portable between operating systems on the same architecture.

IV. Comparison To Other Languages

Feature	Pit	C	Perl
Basic features			
Manual memory allocation	Y	Y	N
Automatic memory allocation & GC	Y	N	Y
Automatic module interfaces	Y	N	NA
Namespaces	Y	N	Y
Preprocessor	N	Y	N
Language constructs			
Homogeneous arrays	Y	Y	C
Heterogeneous arrays	Y	N	Y
Structs	Y	Y	N
Hashes	Y	N	Y
In-line assembly	Y	Y	N
Exceptions	Y	C	C
High-level features			
Abstract data types	Y	N	Y
Automated compilation	Y	N	Y
Native vector math	Y	N	N
Eval	N	N	Y

Yes, No, Clumsy, Not Applicable

The first step to grasp how a new language fits into everyday programming is to gain an intuitive understanding of how design patterns for programs written in the language differ from those written in other languages, especially the languages that are most similar and influential to its design. A good next step is to examine how programs will be simpler or more complex because of the capabilities and limitations of the language. This helps present the language in the context of practical use.

Comparing Pit to C and Perl is useful because Pit is designed to make C's capabilities and Perl's capabilities easy to use together. This will highlight Pit's additional capabilities, acknowledge its limitations, and provide insight into what Pit code looks like and ways it may reduce the effort required to write code.

A. Manual and automatic memory allocation

This feature is, of course, the focus of this paper. In Pit, the programmer chooses which variables are allocated automatically. If the programmer declares a variable to be the auto type, the memory for it and anything stored in it is allocated automatically, and reference-counting garbage collection is performed automatically, similarly to Perl. If the programmer declares a variable to be a primitive type, he must manually allocate and free memory the same way he would in C. The language is designed to generate code that is predictable and consistent in terms of CPU and memory usage; this is one reason it uses reference-counting garbage collection instead of mark-and-sweep garbage collection.

B. Automatic module interfaces

In C, the programmer must meticulously create a header file for every module. Typically, almost all the information in the header file is duplicated in the module's source file. All of the duplicate information could be generated automatically, and therefore almost all of the time programmers spend maintaining header files for C modules is wasted.

Perl has an unusual approach. When a module is imported, it is literally executed. This is because, roughly speaking, Perl code is compiled at run-time, and declaration statements are "executed". In any case, the net effect is that it supports modules in a conventional sense, but does not require manual interface specifications.

Pit uses "interface files"; when a module is compiled, the compiler generates both the object file containing machine code and an interface file for the module. The contents of the interface file fulfill the same role as the contents of most C header files; it contains only the prototypes for functions and shared variables. In contrast to C, however, the compiler generates them automatically. When distributing a module, the interface file and object file may be distributed without the source file.

C. Namespaces and automated compilation

C simply does not support namespaces, although most of its successors do. Perl supports namespaces quite well in the form of what it calls "packages". Pit supports namespaces also, including support for declaring "private" symbols that are only visible inside the namespace and "public" symbols that are visible everywhere. References to public symbols must include the prefix of the name of their namespace except when they are referred to within their own namespace. Pit implicitly declares a namespace for each module with the name of the file containing it; this aids automated compilation, described below. Namespaces may be nested. A filesystem hierarchy may represent nested namespaces surrounding individual modules, and a module may contain multiple namespaces within the namespace defined by its filename.

When compiling a C program, the programmer must somehow describe each command to execute. The compiler must be invoked for each module of the program. Then the modules must be linked with the necessary libraries to produce the final executable file. Little of this process is automated or standardized. While tools such as make ease this process, it is still a complex process that the programmer must design carefully. Most other languages have a better system.

When compiling a Perl program, compilation starts with the main module. When a module requires another module, it contains the "use" statement, which tells the compiler the file name of the required module. The compiler then takes a detour to compile that module before continuing. This process eliminates header files and makes it easier to move the program between systems, since the process of finding a dependency is contained in the compiler rather than, for example, in a Makefile included with the program's source code.

Pit follows an approach similar to Perl. Because the compiler automatically declares a namespace for each module, and

each symbol is prefixed with the name of its namespace, the compiler can find the source file that contains each symbol. To start compiling, the programmer executes the compiler and specifies the main module. When the compiler encounters an "import" statement, it searches the filesystem for the interface file. If it cannot find one or if it finds one that is out of date, it searches for a Pit source file, generates the interface file from it, and adds it to a list of modules that need to be compiled. After compiling the main module, it compiles the modules in that list following the same scheme recursively. When it finishes, it runs the linker and passes it the name of the main module. Similarly to the compiler, the linker finds object files and libraries on the file system automatically based on the names of the symbols it needs to resolve. This way the system can produce the executable with no manual specification of what commands to run or what source files (besides the main module) to use for input. The language and system libraries and third-party libraries might include the interface and object files without source code. Note that this is okay, because if the interface file and object file for a module are available, the compiler skips compiling that module.

Other languages support similar functionality, but this is a new feature in the realm of operating systems written in C. While other preliminary steps may be necessary to preprocess source code or installation steps may be necessary to copy the program and accompanying files into place, the basic compilation process is automatic and simple.

D. Arrays

Typically, homogeneous arrays are less flexible, but they require less memory because the size and type of each element need only be stored once instead of for each element. C supports homogeneous arrays of any type, but not heterogeneous arrays.

Perl only supports homogeneous arrays of 8-bit integers, as a special case for storing text strings efficiently. Surprisingly, this works well for most applications because the majority of homogeneous arrays can either be treated as atomic 8-bit strings or store elements large enough that the extra information does not have a significant impact on memory usage. However, there are important exceptions; for example, Perl cannot deal with a large array of 16-bit integers or floating-point numbers efficiently, making it unsuitable for rendering graphics. (Perl can still play a major roll in a graphics application by using C libraries to deal with image manipulation or 3D calculations. However, coupling two languages is cumbersome.)

Like Perl, Pit's auto variables cannot store homogeneous arrays, except for strings of 8-bit integers (to accommodate UTF-8 strings). Later work will probably improve auto variables to support homogeneous arrays with any subtype. Pit's primitive arrays, like C, support any primitive subtype. Converting between primitive and auto variables is as easy as assigning one to the other, however if an auto variable containing an array is assigned to a primitive array and there is an element of the wrong type or the primitive array is too small, an exception is thrown. If the string is too large, the extra elements are assigned a null value.

E. Structs and hashes

Structs are, of course, borrowed from C. A struct is exactly the same thing in C and Pit. Perl has no support for proper structs, but a struct can be approximated with a hash, since structs and hashes represent nearly the same thing: A set of names mapped to a set of values. The primary difference between a struct and a hash is that the elements of a struct are static, while elements of a hash can be added or deleted at run-time. In Pit, auto variables cannot store structs; instead, a struct can be approximated with a hash. Primitive variables can be declared with the struct type.

F. In-line assembly

In low-level programming, it is often unavoidable to use in-line assembly code (however undesirable it may be) for interaction with hardware, manually optimizing code, switching tasks or managing memory pages in an operating system, etc. Pit's interface to assembly code is similar to GCC's interface for C.

G. Exceptions

Fundamentally, an "exception" is a branch that can exit several functions simultaneously, usually used for error handling. In most languages, exceptions have additional features that significantly improve their usability, but this is the basic meaning. The best that C has to offer are the setjmp and longjmp symbols. These are quite clumsy, as they have no mechanism for allowing intermediate functions to clean up when an exception occurs, and they require a storage variable to be manually passed to functions that may throw the exception.

Perl supports exceptions also, but its support is clumsy in a different way; error messages in Perl are always in the form of user-friendly strings, for example, "No such file or directory". This means that the only way for a program to trap a particular error is to match the user-friendly string. If the string changes, for example because the user speaks a different language or uses a different operating system, the trap for that particular error breaks. Perl has some libraries to mitigate this problem, but they are not widely used. Pit provides a way to report a dependable identifier for an error, a way to map that identifier to a user-friendly message, and a way for intermediate functions to clean up their workspace when an exception occurs.

H. Abstract data types

C can handle abstract data types in an extremely manual way. Perl supports abstract data types quite well. Pit uses auto variables to support abstract data types in a way similar to Perl. However, the description of the mechanics is too lengthy for this paper.

I. Native vector math

In Pit, arithmetic operations on arrays perform a reasonable vector operation when appropriate. For example, adding two

arrays together produces a third array where each element is the sum of the elements in the same position in the input arrays. Assigning one array to another copies the values as if each individual element were assigned. Adding a scalar to an array adds the scalar to each element in the array. Multiplication and the other arithmetic operators work the same as addition. The benefit of this, besides easy vector math, is that the compiler can easily optimize these operations for the target architecture using the CPU's vector math instructions. In C, optimizing vector math typically requires rewriting code to use architecture-specific libraries.

Unlike C, Pit does not use the name of an array to represent a pointer to the array (rather, it uses the reference operator just like with any other variable). This alleviates the ambiguity as to whether adding two arrays means adding their memory addresses or adding their elements. To do the former, the reference operator is used to get their addresses before the operation.

J. Eval

As a compiled language, dynamic code evaluation is difficult, at best. While it is possible to add this later, no work has been done on this. The language itself does not preclude an eval statement; it is only difficult because the compiler for this language generates stand-alone machine code.

Parsing code during run-time is easy; this requires a library that contains most of the logic of the compiler's phases, such as parsing, optimizing, and code generation. When the compiler encounters an eval statement, it should insert calls to the compiler library. The difficult part is resolving symbol names inside the eval that refer to lexically scoped symbols outside the eval. Currently, the names of these symbols are lost in translation. The compiler needs to be modified to include a table of local variable names in the machine code's output, and there needs to be an API for communicating variable locations and execution results between the eval'd code and the compiled code. This is a brief analysis, so there may be other difficulties to overcome.

V. Future Work

The first work to do next is to write a Pit compiler in Pit. This will both demonstrate that Pit can be self-hosting and improve on the current compiler, which has an overly simple design with no optimizations. It was written for a single purpose, which is to bootstrap the compiler written in Pit.

Then the compiler needs some optimization. Most of the optimization techniques used by C compilers should be useful in Pit. There are additional Pit-specific optimizations. For example, a primitive variable can often be substituted for an auto variable as described above. When work on optimization is complete, it is reasonable to expect Pit programs that use only primitive variables to match the speed of equivalent C programs. Pit programs should always be faster than equivalent Perl programs because when a program written in Perl executes, Perl first compiles it into byte-code, then executes it in a virtual machine. Pit programs should execute significantly faster, even if they only use auto variables, because the code is already compiled at run-time and does not require a virtual machine. This places the performance of Pit programs, at worst, somewhere close to C programs and better than Perl programs.

Next, there needs to be some study of the difference in speed between primitive and auto variables, specifically to determine how high the performance penalty is for using auto variables. This will provide insight into when it's appropriate to use primitive or auto variables when weighing the trade-off of a program's performance versus its security, portability, and difficulty to write. Some rough study on this could be done today on toy examples by emulating the logic for auto variables in C. The examples would be lengthy and inconvenient, but with care they could serve the purpose of giving a rough estimate of the difference in speed between the types of variables.

A later useful branch of work may be to create a compiler that translates Pit code into a byte-code that is suitable for a virtual machine. Like Java, this would allow programs to be portable between CPU architectures. It should be possible to do this in a way that allows many programs to be compiled natively or for the virtual machine without modification.

Another possible branch of work may be adapting existing research on checking C code for security bugs. While this is obviated for code that exclusively uses auto variables, this research may still be useful to minimize security risks in code that uses primitive variables.

VI. Related Work

General concern over computer security is probably as old as wide-spread use of computers for routine tasks. In 1972 James Anderson published a report for the Electronic Systems Division of the US Air Force[3]. The primary concerns of his report were security between users of time-sharing systems and security between computers on a network. For its age, his report is surprisingly relevant to modern computing. Security between users is at least as relevant today as it was in 1972 (even if multiuser machinery has perhaps declined), because most operating systems use it to minimize the threat that applications pose to each other. For example, a server running Apache and MySQL typically executes them with limited privileges as separate users so that if Apache is compromised, MySQL is still protected. The importance of security between computers has grown explosively as the Internet has grown, because nearly all computers are now connected to the Internet. However, it wasn't until the 1990s that the industry widely acknowledged that the numerous subtle security flaws in software affect everyone, not just organizations with especially sensitive secrets such as the US Air Force. Today, security is such a high priority that many users will accept severe performance penalties (especially in situations where faster hardware can compensate) or use software that is significantly inferior in other ways. On the other hand, there is significant successful work on reducing the cost of security, where cost refers to every trade-off – performance, maintainability, capabilities, hardware, etc.

Intense interest in security is still relatively new, and many common languages and programs predate wide-spread con-

cerns about security. It wasn't until 1988 that the first documented exploitation of a buffer overrun appeared in the form of the Morris Worm. Interestingly, the report from Cornell on this incident[10] mentions mutual trust between computer users up to this point, and a desire not to "build walls as high as the sky" to protect against intruders. In today's environment of persistent identity thieves, spammers, bot-nets, pirates, and costly hoaxes it seems strange to be so unconcerned with security. Looking back, it seems obvious that the Morris Worm was a disaster waiting to happen, however at a time before online shopping and personal computers for everyone raised the stakes, it probably wasn't unreasonable to assume attackers could be tracked down and prosecuted the same way they are with physical security breaches.

In 1996, the community's equilibrium was again punctuated by Aleph One. This time, there was no turning back. Computers had permeated the public to the point where too many people were interested in breaking security to ignore the issue any longer, and now Smashing The Stack For Fun And Profit[2] detailed exactly how to take advantage of unchecked buffers with an explanation most novice hackers could understand. This is roughly when intense work started in securing software against dedicated attackers and setting up ways of quickly responding as new security holes are discovered. The pattern of the buffer overrun programming bug has been astonishingly pervasive and persistent. In spite of 12 years of auditing since Aleph One threw open the gates, old buffer overrun bugs are still frequently discovered, and new ones are frequently introduced. Simply learning to write bug-free code has proven to be no more effective at eliminating buffer overrun bugs than at eliminating bugs in general.

Starting roughly 2002[5], [1], attackers started exploiting integer overflow bugs and soon other non-control-data attacks[6]. Exploiting these bugs is a more subtle exercise and varies more between programs, but as it becomes harder for attackers to find buffer overrun bugs, they will undoubtedly focus more on other bugs.

Many tools have been developed that focus on detecting buffer overrun bugs at run-time. These can help developers fix old code and mitigate security risks, but none of the run-time tools can detect a bug without a test case that triggers it, and several types of errors are not reported by any of them, including integer overflow bugs and buffer overrun bugs within a data structure. Dynamic Buffer Overflow Detection[19] surveys some tools in this category including Chaperon, Valgrind, CCured, CRED, Insure++, ProPolice, TinyCC, and StackGuard[7].

Safe Systems Programming Languages[12] surveys SafeC, CCured, Vault, and Cyclone. Cyclone extends C to provide bounds-checking. However, it requires extensive modifications to the code, incurs a significant performance penalty to take advantage of its benefits, and still aborts the program when a bug is detected rather than dealing with memory allocation automatically. It also does not address integer overflow bugs. This represents the disadvantages of automatic memory management with none of the benefits. SafeC similarly protects against buffer overruns with run-time bounds checking, but sacrifices performance to avoid code modifications. Vault is an entirely new language that does not support pointer arithmetic, which makes it unsuitable for kernel code. It also suffers from the disadvantage of requiring the programmer to explicitly track memory allocations while restricting pointers so tightly that it could have automated memory management entirely.

SmashGuard[14] presents a solution that is interesting because it is implemented in hardware so it requires no modification to application code and incurs no overhead. Its disadvantages are it protects only against attacks that target function call return addresses, it requires minor kernel modifications, and of course, requires new hardware.

Transparent Runtime Randomization[17] and Address Obfuscation[4] change the memory layout of applications with each execution. This is effective against most attacks because the attacker generally cannot predict the behavior of the program closely enough to compromise it. However, this does not protect against non-control-data attacks or potential denial-of-service attacks because of program crashes. PointGuard[8] is a tool with a different approach but similar results. Rather than rearranging the application's memory layout, it encrypts pointers when they are in memory so that a corrupted pointer will be decrypted to an unpredictable address before being accessed.

The methods that detect an attack in progress and abort the program do not fix the underlying problem that there is a bug in the software. Detection is a big improvement, and in most programs, this is a "safe" thing to do; although the program behaves in an undesirable way, disaster is averted. In many cases, however, aborting a program is dangerous. For example, an abort in the kernel will crash the computer, resulting in down-time and often lost data. A trapped bug in a database server can cause equally catastrophic results. An attacker may exploit any such bug to deliberately keep a service off-line. While sensitive data is not revealed, the solution is still not good enough.

Vault and Cyclone extend C but make code more complicated, and more complication typically means more errors. It is more desirable to simplify code through abstraction instead, and automated memory management is a powerful tool for this. Automated memory management also addresses non-control-data attacks and integer overflows, which most of the above methods do not consider. However, low-level developers have historically avoided languages that provide automated memory management because these languages also prevent access to raw pointers, in-line assembly, and necessary optimization techniques. (Automated memory allocation includes allocating and freeing memory without explicit statements. It also includes extending buffers to avoid overruns and widening integers to avoid overflows.)

This history suggests the following conclusions. C was designed before security was a major concern, and unfortunately in several ways it is impossible to extend C in ways that would solve the problem once and for all without significant changes to all the code in need of protection. The best way to solve memory allocation bugs is obviously to automate memory allocation, but before low-level developers can use automated memory allocation, there needs to be a language that supports both memory allocation and access to raw pointers. Such a

language not only fixes the allocation bugs but also simplifies much code.

The primary disadvantage to creating a new language is that it requires rewriting code, but no solution has been presented yet that can fully protect a program without either unacceptable complexity, unacceptable cost to efficiency, or loss of access to raw pointers. Full protection includes protecting buffers that are part of a struct, and by design, C allows an array contained in a struct to overflow, and many programs utilize that feature in reasonable ways.

VII. Conclusion

Currently, low-level code is almost exclusively written in C. Pit is a much friendlier language for writing low-level code, and it avoids infrastructure that usually deters programmers from choosing other languages for low-level code because of its high overhead or unpredictable behavior, such as an interpreter, a virtual machine, or a mark-and-sweep garbage collector. There is no performance penalty to using Pit when the programmer chooses to use only primitive variables. When the programmer chooses to use auto variables, there is a cost (how much is yet to be seen), but this cost is mitigated in two ways. First, a decent optimizer will eliminate a substantial amount of the overhead by replacing auto variables with primitive variables, and second, the programmer may choose to explicitly write the code using primitive variables, raw pointers, and manual memory allocation where the benefit is worthwhile. Because of the relation of Zipf's Law to execution time, the penalty of automated memory allocation will not be important in most of the code. When efficiency is necessary, Pit provides an exceptionally easy way to interface auto variables with primitive variables. Perhaps the biggest advantage is Pit's the ability to describe time-critical sections in efficient terms while protecting the rest of the code from buffer overruns, integer overflow bugs, non-control-data attacks, double-free bugs, and other memory allocation bugs. Some existing approaches to detecting buffer overrun attacks can easily be applied to Pit's primitive variables and may mitigate the remaining security risks. Object files generated from modules written in Pit are far easier to move between operating systems, because recompiling is usually not required. These considerations make Pit a superior language for low-level programming.

VIII. Appendix: Grammar

A. Convention

This specification uses BNF, Perl Compatible Regular Expressions (PCRE), and several trivial augmentations to BNF as follows:

- <> surrounds rule names. When empty, represents the null rule.
- "" means literal
- [] means optional (0 or 1 occurrence)
- () means grouping
- * means 0 or more occurrences
- + means 1 or more occurrences
- // means comment

B. Token separation

The compiler uses PCREs to tokenize the input file. The first capture buffer ($1) captures a token, and the second ($2) captures white-space. If the expression doesn't match, the compiler tries the next expression. If all fail, the compiler reports a syntax error and aborts. If an expression captures only white-space, matching restarts after the white-space with the first regular expression. If an expression captures only a token, it passes the token to the parser, then again restarts matching after the token with the first regular expression. No regular expression matches both a token and white-space. (These are copied directly from the compiler's source code, with syntactical decoration removed. Assume the s flag to anchor ^ only at the beginning of a string containing newlines.)

```
// Single-quoted strings
^('[^']*')()

// Double-quoted strings may contain \"
^("(?:\\.|[^\\"])*")()

// Single-line comments (treated like white-space)
^()(//[^\n]*)

// Multi-line comments (treated like white-space)
^()(/\*.*?\*/)

// White-space
^()([ \t\r\n]+)

// Bare-words and numbers
^([A-Za-z0-9\.\_]+)()

// Single special character tokens
^([\(\)\[\]\{\}\,\@\#\$\;])()

// Multi special character tokens
^([\'\~\!\%\^\&\*\-\=\+\\\|\:\<\>\/\?]+)()
```

The above identifies boundaries between tokens, but doesn't identify any meaning for the tokens. First, consider individual tokens. Here the specification again uses PCREs for conciseness, but after this section it is restricted to BNF.

```
<number> ::= <integer> | <float>

<integer> ::= ^[0-9]+$
| ^[A-Z0-9]_[A-Z0-9]+$

<float> ::= ^([1-9A-Z]_)?[0-9A-Z]+
  (\.[0-9A-Z]+)?(e[0-9A-Z]+)?$
```

Note that the rules above allow a preceding digit and underscore. This is the notation for expressing a base besides decimal. Any base between 2 and 36 is acceptable. The base is identified by placing the highest digit for that base before the underscore. Here are some examples of <number>:

- A decimal integer: 3735928559
- Another decimal notation: 9_3735928559
- Binary for 3735928559: 1_11011110101011011011111011101111
- Hexadecimal for 3735928559: F_DEADBEEF
- Base-36 for 3735928559: Z_1PS9WXB
- A decimal float: 32.5
- Another decimal notation: 9_32.5
- Binary for 32.5: 1_100000.1
- Hexadecimal equivalent to 32.5: F_20.8
- Base-36 equivalent to 32.5: Z_W.I

```
<quoted-string> ::= ^'[^']*'$
```

```
| ^"(\\.|[^\\"])*"$
```

Single-quotes do not allow special encodings, so it is simpler to use them for strings with literal back-slashes, but it is impossible express single-quotes within a string identified by single-quotes. Double-quotes allow back-slash encodings such as \" \\ \n etc. Encoding is the only difference between the quoting types. The compiler is 8-bit clean, so a UTF-8 string may appear in quotes with no special support.

```
<bare-word> ::= ^[A-Za-z0-9_.]+$

<symbol> ::= "$" <bare-word>
```

<symbol> is any entry in the symbol table (any variable or function) defined by source code. Note that the leading $ obviates the need for reserved symbol names. Language keywords and numbers such as 32, F_20, if, and asm are distinct from variables and functions named $32, $F_20, $if, and $asm. This allows future versions of the language to include new keywords without breaking old code, which is a problem that C suffers from frequently. While this means there are no reserved symbol names, names should not begin with an underline because these may be used internally by the compiler tools. For example, the symbol "_GLOBAL_OFFSET_TABLE_" is used on many platforms for dynamic linking.

```
<type-def> ::= "@" <bare-word>
```

<type-def> is any name defined by source code to refer to a particular data type. Terminologically, a <symbol> has an associated <type-expression> (which can be a <type-def>), and a <type-def> is a short-hand name for a <type-expression>. Admittedly, both are tracked in the compiler's symbol table, so this specification can be somewhat confusing distinction if unexplained. <symbol>s are variables and functions. <type-def>s are user-defined types for <symbol>s. As with symbols, names beginning with an underline should be avoided.

```
<jump-label> ::= "#" <bare-word>
```

A jump-label is any name defined by source code to refer to an entry point for execution. These are used for exception handling and for arbitrary jumps. (Pit uses the keyword "jump" instead of "goto" for consistency with assembly language.) The scope of a jump-label is limited to the function it is defined in. These use # instead of $. These labels may be used in expressions to represent a memory address of executable code, which is useful for in-line assembly code. Again, names beginning with an underline should be avoided. The compiler will almost always need to define jump labels for flow control constructs; these are always named with an underline prefix so they don't conflict with user-defined jump-labels. Unlike GCC, user-defined jump-labels are not renamed because in-line assembler code may need to reference them.

C. Modules and statements

```
<module> ::= <import-statement>* <statement>*
```

An entire source file is a <module>. <import-statement>s must come before other <statement>s.

```
<import-statement> ::= "import" <bare-word> ";"
```

<import-statement>s simply reference other libraries. Dots in library names form a hierarchical naming convention corresponding to namespaces and subdirectories in the library search path.

```
<statement> ::= <namespace-statement>
| <asm-statement>
| <conditional-statement>
| <loop-statement>
| <exception-statement>
| <declaration-statement>
| <expression> ";"
| "return" ";"

<statement-block> ::= <statement>
| "{" <statement>* "}"

<namespace-statement> ::= "namespace" ":" <statement-block>
| "namespace" <bare-word> ":" <statement-block>
```

A namespace may be anonymous (without a name). Anonymous namespaces have little effect except that the private symbols in them may only be accessed by other symbols within the anonymous namespace. Visibility of public symbols is unaffected by an anonymous namespace. A named namespace has the same effect on private symbols, plus the effect that contained public symbols are all prefixed with the name of the namespace and a dot.

<asm-statement> isn't defined yet. It will represent syntax for specifying in-line assembler code.

```
<conditional-statement> ::= ("if" | "unless") "("
  <expression> ")" <statement-block> ["else"
  <statement-block>]

<loop-statement> ::= "loop" [<bare-word>] ":"
  <statement-block> ["iterate" <statement-block>]
| "again" ";"
| "next" ";"
| "break" ";"

<exception-statement> ::= "try" "#" <bare-word>
  <statement-block>
| "throw" <expression> ";"
| "label" "#" <bare-word> ";"
| "jump" <expression> ";"

<declaration-statement> ::=
  <visibility> <type-expression> <type-def> ";"
| <visibility> <type-expression> <symbol> ";"
| <visibility> <type-expression> <symbol> "prototype" ";"
| <visibility> <type-expression> <symbol> "alias" "=" "$"
  <bare-word> ";"
| <visibility> <type-expression> <symbol> "=" <initializer>

<visibility> ::= "public"
| "private"
```

<declaration-statement>s define both variables and functions. Functions are declared using initializer syntax, since functions are just constant symbols that contain executable code. Functions are generally a bit of an enigma to syntax, although they are less so in Pit than in most other languages. For example, the sizeof operator does not know how to figure out their size; they are always constant data; the function initializer can only be used in a declaration-statement, not in an generic expression; etc.

D. Type-expressions

```
<type-expression> ::= <type-expression-bracket>
| <type-expression-bare>
```

```
<type-expression-bracket> ::= "[" <type-expression-bare> "]"

<type-expression-bare> ::= <type-qualifier>*
  <type-expression2> <type-expression3>

<type-qualifier> ::= "signed"
| "unsigned"
| "const"
| "unconst"
| "volatile"
| "unvolatile"
| "stateless"
| "unstateless"

<type-expression2> ::= "auto"
| "char"
| "int" ["(" <integer> ")"]
| "ptr" ["(" <type-expression> ")"]
| "float" ["(" <integer> ")"]
| "string" "(" <type-expression> ["," <integer>] ")"
| "struct" "(" (<type-expression> <quoted-string> ",")+ ")"
| "function" "(" (("in" | "out" | "inout") <type-expression>
  "$" <bare-word> ",")* ")"
| "@" <bare-word>  // user-defined type
| "$" <bare-word>  // type of another symbol

<type-expression3> ::= <>
| "->" <type-expression3>
| "<-" <type-expression3>
| "(" ")" <type-expression3>
| "(" <quoted-string> ")" <type-expression3>
| "(" <symbol> ")" <type-expression3>
```

Type-expressions are used for declaring and prototyping symbols, type-casting, within other type-expressions, and the sizeof operator. Note that two of the alternatives in <type-expression3> allow referencing the type of an element in a struct or a parameter to a function. Square brackets are used exclusively to identify a type-expression. For example, a type-cast converting $foo to a 128-bit integer would look like this:

```
$foo[int(128)]
```

In certain cases where the syntax requires a type-expression, the square brackets are required (<type-expression-bracket> is used for these). Specifically, they are required by type-casts and the sizeof operator; they are not required by declaring/prototyping symbols or within other type-expressions.

E. Expressions

```
<expression> ::= <value> [<binary-op> <expression>]

<value> ::= <unary-op> <value>
| <value> <postfix-unary-op>
| "sizeof" <type-expression-bracket>
| "(" <expression> ")"
| "(" "if" <expression> "then" <expression> "else"
  <expression> ")"
| <symbol>
| "error_id"
| "line_number"
| <number>
| <quoted-string>
| <jump-label>
| ["array" | "hash"] "{" (<immediate> ";")* "}"

<unary-op> ::= "-" | "~" | "not"

<binary-op> ::= "=" | "+=" | "-=" | "*=" | "/=" | "/-="
| "/+=" | "/<=" | "/>=" | "%=" | "|=" | "~|=" | "^=" | "~^="
| "&=" | "~&=" | "<<=" | ">>=" | ":" | "+:" | "-:" | "*:"
| "/:" | "/-: | "/+:" | "/<:" | "/>:" | "%:" | "|:" | "~|:"
| "^:" | "~^:" | "&:" | "~&:" | "<<:" | ">>:" | "or" | "nor"
| "xor" | "nxor" | "and" | "nand" | "<" | "<=" | "==" | "!="
| ">=" | ">" | "<=>" | "min" | "max" | "+" | "-" | "*" | "/"
| "/-" | "/+" | "/<" | "/>" | "%" | "|" | "~|" | "^" | "~^"
| "&" | "~&" | "<<" | ">>"

<postfix-unary-op> ::= "->"    // dereference operator
| "<-"                         // reference operator
| <type-expression-bracket>    // typecast
| "(" <expression> ")"         // element of a string
| "(" <quoted-string> ")"      // element of a for struct
| "(" (<expression> ",")* ")"  // function call
```

F. Notes

The grammar is mostly presented in an order that minimizes the number of unexplained rules at each point as read from top to bottom. For example, before defining <expression> and referencing many new undefined rules, the grammar ties off all the other loose ends.

One minor but intriguing invention is the syntax for numbers in bases besides decimal. This allows easy representation of any base up to 36, and avoids the confusion caused by the common convention of using a leading zero to designate octal.

At the moment, for-loop syntax is not included. There is some debate over the best syntax for this. Rather than regretting a bad decision later, this matter is unfinished for the time being. Meanwhile, the <loop-statement> syntax allows a clause called “iterate” that mostly covers what for-loops can do.

As presented, the grammar appears to support function declarations within functions, however the compiler does not implement this. Also, there are restrictions as to which alternatives for <statement> are allowed inside a function and which are allowed outside a function. This reflects how the proof-of-concept compiler is implemented; code generation, not parsing, enforces these restrictions.

Some details in this grammar still need attention, however there is a functioning compiler that translates the grammar as presented here into i386 assembly language with support for auto variables.

REFERENCES

[1] Dave Ahmad. The rising threat of vulnerabilities due to integer errors. *IEEE Security and Privacy*, 01(4):77–82, 2003.

[2] Aleph One. Smashing the stack for fun and profit. *Phrack http://www.phrack.org/issues.html?issue=49&id=14*, 7(49):14, 1996.

[3] J. P. Anderson. Computer security technology planning study. *Air Force Electronic Systems Division ESD-TR-73-51, Vols. I and II*, 1972.

[4] Sandeep Bhatkar, Daniel C. DuVarney, and R. Sekar. Address obfuscation: An efficient approach to combat a broad range of memory error exploits. In *12th USENIX Security Symposium*, August 2003.

[5] Blexim. Basic integer overflows. *Phrack http://www.phrack.org/issues.html?issue=60&id=10*, 11(60):10, 2002.

[6] Shuo Chen, Jun Xu, Emre C. Sezer, Prachi Gauriar, and Ravishankar K. Iyer. Non-control-data attacks are realistic threats. In *14th USENIX Security Symposium*, pages 177–192, 2005.

[7] Crispan Cowan, Calton Pu, Dave Maier, Jonathan Walpole, Peat Bakke, Steve Beattie, Aaron Grier, Perry Wagle, Qian Zhang, and Heather Hinton. StackGuard: Automatic adaptive detection and prevention of buffer-overflow attacks. In *7th USENIX Security Conference*, pages 63–78, January 1998.

[8] Crispin Cowan, Steve Beattie, John Johansen, and Perry Wagle. PointGuard™: Protecting pointers from buffer overflow vulnerabilities. In *12th USENIX Security Symposium*, August 2003.

[9] Crispin Cowan, Perry Wagle, Calton Pu, Steve Beattie, and Jonathan Walpole. Buffer overflows: Attacks and defenses for the vulnerability of the decade. In *DARPA Information Survivability Conference & Exposition – Volume 2*, pages 119–129, January 2000.

[10] T. Eisenberg, D. Gries, J. Hartmanis, D. Holcomb, M. S. Lynn, and T. Santoro. The cornell commission: on morris and the worm. *Commun. ACM*, 32(6):706–709, 1989.

[11] FreeBSD. FreeBSD ports collection. http://www.freebsd.org/ports/.
[12] Peng Li. Safe systems programming languages, October 2004.
[13] OpenBSD. OpenBSD security. http://openbsd.org/security.html.
[14] H. D Özdoğanoğlu, T. N. Vijaykumar, C. E. Brodley, A. Jalote, and B. A. Kuperman. Smashguard: A hardware solution to prevent security attacks on the function return address, November 2003.
[15] Leif Pedersen. Pit programming language. http://pit.devpit.org/.
[16] Leif Pedersen and Hassan Reza. A formal specification of a programming language: Design of Pit. In *ISOLA '06: Proceedings of the Second International Symposium on Leveraging Applications of Formal Methods, Verification and Validation (isola 2006)*, pages 111–118, Washington, DC, USA, 2006. IEEE Computer Society.
[17] Jun Xu, Zbigniew Kalbarczyk, and Ravishankar K. Iyer. Transparent runtime randomization for security. *srds*, 00:260, 2003.
[18] Jun Xu, Zbigniew Kalbarczyk, Sanjay Patel, and Ravishankar K. Iyer. Architecture support for defending against buffer overflow attacks. In *Workshop on Evaluating and Architecting System Dependability (EASY)*, October 2002.
[19] Michael Zhivich, Tim Leek, and Richard Lippmann. Dynamic buffer overflow detection. In *Workshop on the Evaluation of Software Defect Detection Tools 2005*, 2005.